LostLink: Campus Lost and Found Management System

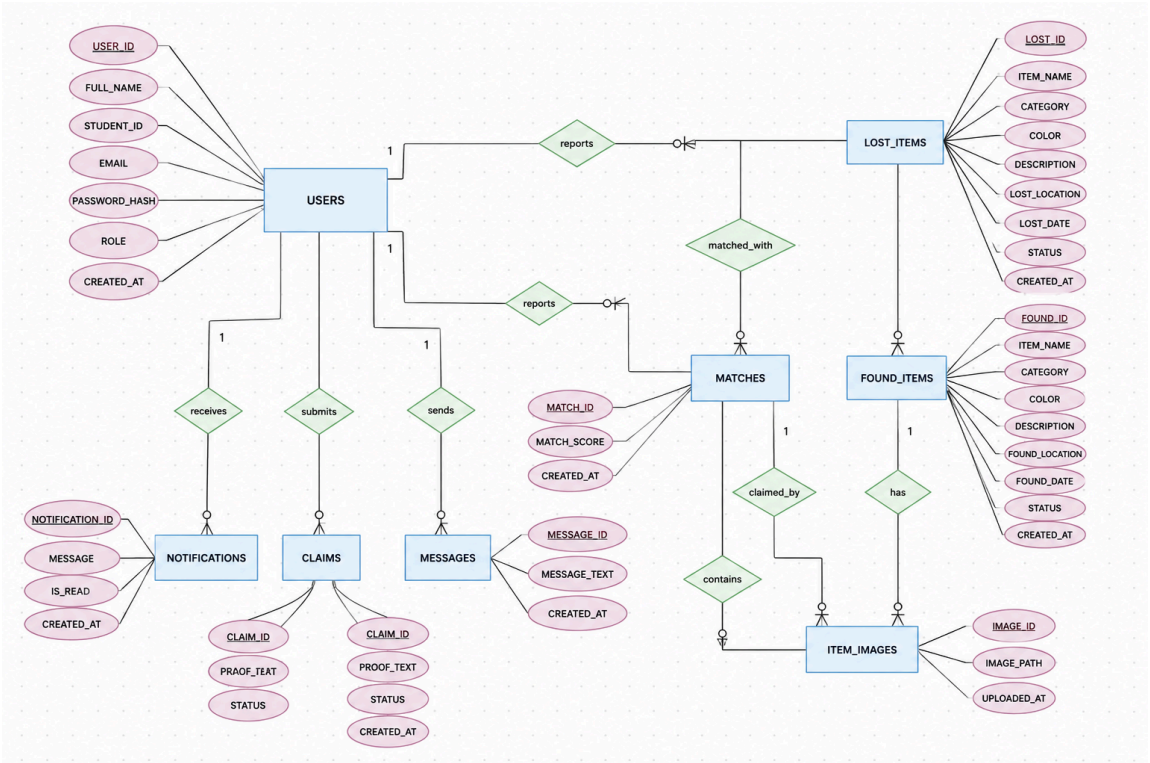
Project Overview

The LostLink database system manages the backend operations for a centralized campus lost and found platform. The system handles user authentication, reporting of lost and found items, smart matching, secure claim processing, and real-time notifications. The system requires strict transaction management to maintain data integrity and prevent false claims. The database was designed to facilitate connections between finders and losers while keeping an auditable log of activities. The application interface helps to interact with the database layers efficiently.

Tools & Technologies

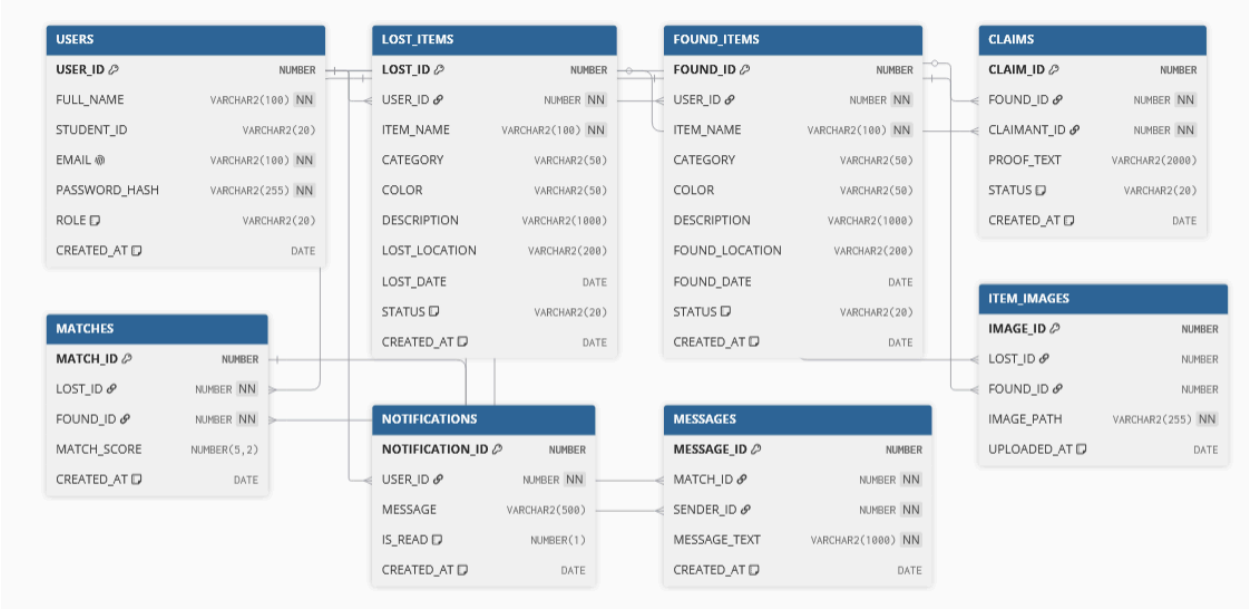
- **Database Engine**: Oracle Database 11g Express Edition (XE)
- **Query Language**: SQL and PL/SQL
- **Web Application**: PHP 8 (OCI8 Extension), HTML, CSS, JavaScript
- **Web Server**: Apache (XAMPP)

ER Diagram



Relational Schema Diagram

The Relational Schema Diagram represents the actual physical tables, primary keys, and foreign key dependencies implemented in the database.



Database Tables

USERS Table

The USERS table stores central authentication credentials for all system participants. The database needs this table to centralize login access and role management. A default role of 'USER' is applied, separating regular participants from system administrators.

LOST_ITEMS Table

The LOST_ITEMS table maintains records of items reported as lost by users. It includes details such as item name, category, color, description, and the location it was lost. A foreign key links each lost item to the user who reported it.

FOUND_ITEMS Table

The FOUND_ITEMS table stores information about items that have been found and reported on the campus. Similar to lost items, it contains descriptive details and location data. The table contains a status column that defaults to 'UNCLAIMED'.

ITEM_IMAGES Table

The ITEM_IMAGES table manages multimedia attachments. It stores file paths for uploaded images of either lost or found items, acting as visual proof. It references both the LOST_ITEMS and FOUND_ITEMS tables using nullable foreign keys.

MATCHES Table

The MATCHES table records automated or manual pairings between a lost item and a found item. It calculates and stores a MATCH_SCORE to indicate the confidence level of the pairing. It references both LOST_ITEMS and FOUND_ITEMS.

CLAIMS Table

The CLAIMS table works as the secure ledger for users attempting to recover an item. It links a FOUND_ID with the user making the claim (CLAIMANT_ID) and requires textual proof of ownership (PROOF_TEXT). The status defaults to 'PENDING' until an admin verifies it.

NOTIFICATIONS Table

The NOTIFICATIONS table stores system-generated messages delivered to users. The database uses this table to notify users about smart matches, claim updates, and chat messages.

MESSAGES Table

The MESSAGES table enables communication between users connected via a smart match. It holds the actual chat messages, referencing the MATCH_ID and the SENDER_ID to ensure secure and context-aware messaging.

Relationships

One-to-Many Relationships

The USERS table maintains a one-to-many relationship with multiple tables including LOST_ITEMS, FOUND_ITEMS, CLAIMS, NOTIFICATIONS, and MESSAGES. A single user can report multiple items or send multiple messages, but each item/message belongs to only one user.

Many-to-Many Relationships

The relationship between Lost Items and Found Items is many-to-many, as one lost item could potentially match multiple found items and vice versa. This was resolved using the MATCHES table as an intersection entity connecting LOST_ID and FOUND_ID.

Stored Procedures

Stored procedures handle all the primary transactional workflows in the system. They encapsulate the business logic, ensuring secure and atomic database operations.

- **SP_REGISTER_USER**
  Purpose: Handles new user registration.
  Description: Securely inserts a new user record into the USERS table with their full name, student ID, email, and password hash. It abstracts the account creation logic from the PHP layer.
- **SP_REPORT_LOST_ITEM & SP_REPORT_FOUND_ITEM**
  Purpose: Submits new lost and found item records.
  Description: Safely inserts details (name, category, color, description, location, date) into LOST_ITEMS and FOUND_ITEMS. It utilizes OUT parameters to immediately return the newly generated LOST_ID or FOUND_ID, which is crucial for the subsequent image upload process.
- **SP_FILE_CLAIM**
  Purpose: Creates a new ownership claim.
  Description: Inserts a new record into the CLAIMS table when a user attempts to claim a found item, setting the initial status to PENDING and recording their proof of ownership.
- **SP_PROCESS_CLAIM**
  Purpose: Admin claim verification.
  Description: Used by administrators to APPROVE or REJECT a pending claim. If approved, this procedure automatically cascades the status updates,

marking the FOUND_ITEM and its corresponding LOST_ITEM as RECOVERED.

- **SP_UPDATE_LOST_ITEM & SP_DELETE_LOST_ITEM**
  Purpose: Item management.
  Description: Allows users to update the details of their active lost items or securely delete a reported item from the system if they no longer need the listing.
- **SP_UPDATE_USER_PROFILE**
  Purpose: Profile management.
  Description: Updates user credentials, specifically the full name and student ID in the USERS table.
- **SP_ADD_MESSAGE**
  Purpose: Chat system insertion.
  Description: Securely inserts real-time chat messages between matched users into the MESSAGES table.
- **SP_ADD_ITEM_IMAGE**
  Purpose: Handles multimedia metadata.
  Description: Links the uploaded image file path directly to either a LOST_ID or FOUND_ID in the ITEM_IMAGES table.

Functions

Functions are used primarily for data retrieval, analytics, and complex calculations. Many of these functions return a SYS_REFCURSOR, allowing the PHP frontend to fetch dynamic sets of data.

- **FN_GET_USER_BY_EMAIL**
  Purpose: Authentication lookup.
  Description: Retrieves a user's details and password hash based on their email during the login process.
- **FN_CALCULATE_MATCH_SCORE**
  Purpose: Smart matching logic.
  Description: The core algorithm that evaluates the similarity between a lost item and a found item. It calculates a numeric score (e.g., comparing category, color, and date) to determine if the items are a potential match.
- **FN_GET_USER_CLAIMS & FN_GET_PENDING_CLAIMS**
  Purpose: Claim retrieval.
  Description: FN_GET_USER_CLAIMS returns all claims specific to a claimant. FN_GET_PENDING_CLAIMS is used by the admin dashboard to retrieve a list of all claims awaiting verification across the system.

- **FN_GET_USER_NOTIFICATIONS**
  Purpose: Alert retrieval.
  Description: Fetches unread and historical notifications for a specific user.
- **FN_GET_MESSAGES**
  Purpose: Chat retrieval.
  Description: Retrieves the chronological chat history for a specific smart match between users.
- **FN_GET_ITEM_IMAGE**
  Purpose: Media utility.
  Description: A scalar function that takes a LOST_ID or FOUND_ID and returns the file path of the associated primary image, making it easy to display images in standard SQL queries.
- **FN_GET_USER_PROFILE**
  Purpose: Profile data retrieval.
  Description: Retrieves all necessary display information for the user's profile page.
- **FN_GET_OVERALL_STATS, FN_GET_CATEGORY_STATS,**
  FN_GET_HOTSPOT_STATS, FN_GET_RECENT_ACTIVITY
  Purpose: Dashboard Analytics.
  Description: These cursor-returning functions calculate and aggregate the metrics used on the admin and user dashboards (e.g., total items, items by category, most frequent lost locations, and recent system actions).

Triggers

Triggers automatically execute in the background based on specific database events, ensuring strict data integrity and automating workflows without requiring application-level logic.

- **Auto-Increment Triggers (Sequence Binding)**
  Purpose: Primary Key Generation.
  Description: Since Oracle 11g lacks native AUTO_INCREMENT, these BEFORE INSERT triggers intercept inserts and fetch the NEXTVAL from their respective sequences (e.g., USERS_BI, LOST_ITEMS_BI, FOUND_ITEMS_BI) to assign unique primary keys automatically.
- **TRG_MATCH_ON_LOST_INSERT**
  Purpose: Automated lost item matching.
  Description: Fires immediately after a new LOST_ITEM is reported. It triggers the matching engine to scan all existing UNCLAIMED found items, calculate

match scores, and instantly populate the MATCHES table if similarities are found.

- **TRG_MATCH_ON_FOUND_INSERT**
  Purpose: Automated found item matching.
  Description: Fires immediately after a new FOUND_ITEM is reported. It scans all ACTIVE lost items, calculates scores, and inserts pairing records into the MATCHES table to proactively alert users that their item may have just been found.

System Workflow

Phase 1 — User Registration and Item Reporting

1. Citizens register as users using their Name, Student ID, and Email.
2. The system validates email uniqueness and prevents duplicate registrations.
3. Users report lost or found items, filling out comprehensive details including category, location, and date.
4. Users upload images of the items, which are securely stored and referenced in the ITEM_IMAGES table.

Phase 2 — Smart Matching and Communication

1. The system algorithmically identifies potential matches between lost and found items.
2. A MATCHES record is created with a calculated match score.
3. Notifications are generated alerting users of potential matches.
4. Users utilize the MESSAGES table to chat securely regarding the matched items to verify details.

Phase 3 — Claims and Verification

1. Once a user identifies their item from the Found Registry, they submit a formal claim with proof of ownership.
2. The claim status is set to 'PENDING'.
3. Administrators log into the system and review the claim and proof text.
4. Admins change the claim status to 'APPROVED' or 'REJECTED'.
5. Upon approval, the item's status is updated to 'Recovered'.

Phase 4 — Analytics & Reporting

1. Dynamic dashboards calculate and display Total Lost, Total Found, Recovered, and Pending Claims.
2. Regular users see statistics restricted to their personal submissions.
3. Administrators view global, system-wide metrics.

System Features

- **User Authentication**: Secure hashing and role-based access control (Admin vs User).
- **Smart Matching System**: Automated pairing of lost and found items based on categories and details.
- **Real-Time Communication**: Integrated chat option allowing users to discuss matches directly.
- **Search & Filtering**: Real-time JavaScript filtering by category and status on the frontend.
- **Dynamic Dashboards**: Context-aware statistic cards that adapt based on the user's role.
- **Image Management**: Seamless upload and retrieval of item imagery for visual verification.
- **Secure Claims Processing**: Admin-gated claim verification process to prevent false ownership assertions.

Conclusion

The LostLink database successfully provides a secure backend for campus lost and found management. By implementing the backend logic directly inside Oracle using PL/SQL, the system guarantees data consistency using the application interface. The strict relational design ensures that data anomalies and redundancy don't occur. The use of robust foreign keys, triggers, and sequences handles complex workflows like smart matching and claim verification efficiently.